Submitting to F-Droid Quick Start Guide

This is a step-by-step guide for packaging an app for F-Droid. It is apparently the best way to get an app into the repository, since you can provide a direct merge request for the metadata about your app, making it easy for the maintainers.

Prepare and compliance check

Before proposing inclusion of an application, the compliance with the Inclusion Policy should be checked carefully. Here is a short checklist.

1. The app should have a public source code repository and a FOSS license file. Please confirm that the repo has the real up-to-date source code in it instead of some dummy files as placeholder.
2. The app should only have FOSS dependencies. Firebase and GMS are some notable examples of non-FOSS libs that are not accepted. If the app can work (in some capacity) without these, please create a flavor/version without them. The build tools should also be FOSS. If a proprietary IDE is required then it can't be included. F-Droid builds from command line tools anyway, build instructions help contributors.
3. The author of the app has been notified (and does not oppose the inclusion). If you are not the author, please open an issue in the repo of the app, asking the author for the permission.
4. The metadata files for descriptions are added to the repo. This is a simple structure with some text files and pictures and should always be added before inclusion. While the folder structure follows Fastlane/Triple-T, the actual tooling is not needed.

With all these requirements satisfied, this app should be ready for inclusion.

Upstream metadata

Each official release commit in the application's upstream Git repository should have a tag. For example, if its AndroidManifest contains `versionName: '1.0'`, the commit needs a `v1.0` tag. It is strongly encouraged to add metadata in the application's source repo, too:

- *fastlane/metadata/android/en-US/short_description.txt* (less than 80 characters, no trailing dot)
- *fastlane/metadata/android/en-US/full_description.txt*
- *fastlane/metadata/android/en-US/images/icon.png*
- *fastlane/metadata/android/en-US/images/phoneScreenshots/1.png*
- *fastlane/metadata/android/en-US/images/phoneScreenshots/2.png*

If the AndroidManifest contains `versionCode: 123`, there should be a corresponding explanation of what's new in this version:

- *fastlane/metadata/android/en-US/changelogs/123.txt* (max 500 characters)

Browse the F-Droid client metadata directory to see a real-world example.

Other formats and locations of the description, graphics, and screenshots are supported, too. For example, the *metadata/en-US* directory can instead be *fastlane/metadata/android/en-US*.

Send the application's development team a merge request if the repository doesn't have those files, and open an issue if it has no version tags. Having this metadata in place lets it be under direct control of the developer, and their updates and future translations will be pulled automatically.

Understand the build metadata

Let's take a look at the build metadata of the official F-Droid client as example. The file lives at https://gitlab.com/fdroid/fdroiddata/-/blob/master/metadata/org.fdroid.fdroid.yml. The fdroiddata repo contains the Build Metadata for all the apps of the F-Droid main repository in the `metadata` folder.

The fdroidserver is used to process these metadata and run the build. The file name of the metadata, i.e. `org.fdroid.fdroid`, corresponds to the application id of the app.

The first part of the metadata is totally descriptive, including `AntiFeatures`, `Categories`, `AuthorName`, `AuthorEmail`, `AuthorWebSite`, `License`, `WebSite`, `SourceCode`, `IssueTracker`, `Translation`, `Changelog`, `Donate`, `Liberapay`, `OpenCollective`, `Bitcoin`, and `Litecoin`. These information is displayed in the client and website.

The second part controls how to build the apk. It includes:

- `RepoType` and `Repo`, the metadata of the VCS of the source code
- `Builds`, a list of build blocks each of which corresponds to an apk which has been built or will be built
- `Binaries` and `AllowedAPKSigningKeys`, the metadata of the reference binary for reproducible build

When there is a new build block in `Builds`, *fdroidserver* start a build according to it. For example, the following build block defines such a process:

```
- versionName: 1.23.2
  versionCode: 1023052
  commit: 0ea06b1c1d68765c4a434d0b676b3e0f23f04cb1
  subdir: app
  gradle:
    - full
  scandelete:
    - app/src/androidTest/assets
    - app/src/test/resources
    - libs/sharedTest/src/main/assets
    - libs/index/src/commonTest/resources
  gradleprops:
    - strict.release
```

First, the source code is cloned from the `Repo`. Then the `commit` is checkout. The work directory is set to `app`. Before build, the scanner runs on the source code and binary files in the paths defined in `scandelete` is removed. Then *fdroidserver* runs Gradle command to build the app:

```
gradle assembleFullRelease -Pstrict.release
```

The apk is found in the default path of a Gradle project and it's checked to ensure it has the correct `versionName` and `versionCode`. Finally, this apk will be signed and published along with other apks.

The last part is about how we should update the app, including `AutoUpdateMode`, `UpdateCheckMode`, `UpdateCheckIgnore`, `VercodeOperation`, `UpdateCheckName`, `UpdateCheckData`, `CurrentVersion`, and `CurrentVersionCode`. The checkupdates runner runs everyday, checking the repo and finding new versions. If there is a new version, the metadata is updated and a merge request will be opened.

More details are available about every field of the build metadata in the Build Metadata Reference and Repository Style Guide.

Write the build metadata for an Gradle app

In the following steps, we assume that the application id of your app is `com.example`. Please use a unique id for you app, corresponding to your domain name. Fork `fdroiddata`, clone the repo and create a new branch from master, e.g., `com.example`. There are more details in https://gitlab.com/fdroid/fdroiddata/-/blob/master/CONTRIBUTING.md. It's also recommended to have fdroidserver installed when you write the metadata. It can help you format and lint the metadata. Create the metadata file at `metadata/com.example.yml`. Open it in your editor and let's start writing your metadata. You can also start from a template. Since there are thousands of build metadata files you generally can find similar build metadata which can be used as a base of your own metadata. E.g., if your app uses Flutter you can search Flutter apps `fdroiddata` repo. There are also examples in the templates folder. You can also use `fdroid import` to generate a template with *fdroidserver*.

Now you have an empty metadata file or a simple template. Please provide as much as possible descriptive information so that the user can know the app and its author better.

Then add the `Repo` info and a build block in `Builds`. For Gradle apps this is simple. And popular toolchains, e.g., Flutter are also covered by our templates. If your app is built with other toolchains and you can't find example in *fdroiddata*, you can define all the build

steps with `sudo`, `prebuild` and `build` manually. The `output` must be set so that *fdroidserver* can find the apk.

Autoupdate configuration

Unless you have a special reason to control the update manually, you should setup auto update. This reduces the maintaince cost for both F-Droid maintainers and you. With auto update enabled, you just need to bump the version and tag a new release. F-Droid will check your repo regularly and update the metadata when there is a new version found. For Gradle apps with the `versionName` and `versionCode` in the normal location, i.e., the `android` block or the `AndroidManifest.xml`, no special setup is needed. But if you put the version info somewhere else, you need to extract them with `UpdateCheckData`. The version info can't be composed or calculated dynamically since F-Droid only uses regex to extract them and won't run the Gradle code.

Setup ABI split

It's not a hard requirement to setup ABI split but if the apk is large and ABI split can reduce the apk size effectively, this is highly encouraged.

Currently there is no special support for ABI split. So every apk should be added as a build block, with different version code and build steps. You can use different Gradle flavors or properties to build different native libs. You can also patch the code in `prebuild` to control the ABI. These apks with different ABIs will be built one by one. Please note that the version codes of different ABIs must be set specially. F-Droid clients always updates the app to the apk with the highest version code that can be installed on the device, so generally the version codes should have such an order: armeabi-v7a < arm64-v8a < x86 < x86_64. Because *fdroidserver* only keeps the apks with the highest version codes in the `repo` and others are moved to `archive`, the version codes of a new version must be higher than the version codes of an old version. In other words, the digits representing the ABI must be put at the lowest position of the version codes. The `VercodeOperation` needs to be set to calculate the version codes, e.g.

```
VercodeOperation:
  - 10 * %c + 1
  - 10 * %c + 2
```

```
- 10 * %c + 3
- 10 * %c + 4
```

Setup Reproducible Build

Reproducible builds are not a requirement for apps being on F-Droid. But we do consider their use best practice. And unfortunately, one can't easily switch to them later because Android doesn't allow updates with a different signing key, meaning users would have to reinstall. So we mainly encourage their use for new apps.

The point of reproducible builds is that the developer's signature (from the APK they publish) guarantees that our build is identical to theirs (and thus doesn't contain anything it shouldn't) and at the same time our build server verifies that the developer's build matches the published source code (and thus doesn't contain anything it shouldn't either).

This increases trust and makes supply-chain attacks harder. It also makes it impossible for there to be a bug in the F-Droid version only (or vice versa). Using the developer's key also means they have the option of providing updates to users themselves if we for some reason (temporarily) cannot.

Some apps – especially those without native code, using only Kotlin/Java – are very easy to make reproducible. Others may require more work. Sadly, some apps cannot be made reproducible at all.

We hope that developers agree with us that it's at least worth attempting to make their apps reproducible given the various benefits, but if they are unable or unwilling to spend time/resources on this, we of course respect their decision.

For more information, see:

- Towards a reproducible F-Droid
- F-Droid's Reproducible Builds documentation
- Reproducible Builds project
- HOWTO: diff & fix APKs for Reproducible Builds

Test the metadata

The initial build metadata may not work as expect. We need to test it. If you have fdroidserver installed, run `fdroid lint <appid>` to get some hints about general issues and `fdroid rewritemeta`

`<appid>` to format the metadata file. Then we can try building the apk with the metadata. If you want to go the hard way, please read the doc about how to run the build locally. Instead, you can use our CI to test your metadata easily. Commit and push your changes to your fork. The pipelines will be triggered automatically. If all the pipelines pass, congratulations! You have a working build metadata now. If some pipelines fail, please read the log and fix the metadata accordingly.

Build environment

On a laptop with Ubuntu 21.10 in February 2022, it took 2 GB of traffic and 5 GB of disk space to set up an F-Droid build environment.

Network requirements:

- 60 MB: shallow-clone *fdroiddata* and *fdroidserver*
- 75 MB: install *docker.io*
- 1000 MB: load the container
- 800 MB: build

Storage requirements:

- 1000 MB: clone the repos and install *docker.io*
- 4000 MB: load the container and build

Download and launch the latest version of the server tools container:

```
git clone --depth=1 https://gitlab.com/fdroid/fdroidserv
sudo sh -c 'apt-get update &&apt-get install -y docker.i
sudo docker run --rm -itu vagrant --entrypoint /bin/bash
  -v ~/fdroiddata:/build:z \
  -v ~/fdroidserver:/home/vagrant/fdroidserver:Z \
  registry.gitlab.com/fdroid/fdroidserver:buildserver
```

In the container:

```
. /etc/profile
export PATH="$fdroidserver:$PATH" PYTHONPATH="$fdroidser
export JAVA_HOME=$(java -XshowSettings:properties -versi
cd /build
fdroid readmeta
fdroid rewritemeta com.example
fdroid checkupdates --allow-dirty com.example
```

```
fdroid lint com.example
fdroid build com.example
```

If any command, such as `fdroid readmeta`, returns an error, edit *~/fdroiddata/metadata/com.example.yml* accordingly and try running the command again. After a successful build, exit the container, commit your metadata file with a `New App` label, and push it to your fork:

```
exit
cd ~/fdroiddata
git add metadata/com.example.yml
git commit -m "New App: com.example"
git push origin com.example
```

Create a merge request at the *fdroiddata* repository, selecting your *com.example* source branch. Wait for the packagers to pick up your merge request. Please keep track if they asked any questions and reply to them as soon as possible.

Troubleshooting

You can get help with F-Droid via IRC, Matrix, XMPP, e-mail and some other channels.

Application Review Process

Once the inclusion proposal is filed, the application will enter a reviewing process where F-Droid staff look into the applications source code and determine whether it fits for inclusion (and when it's not, determine all necessary steps to make it so).

As F-Droid is a software repository which promises users free software, a review process is for ensuring that all applications distributed from the F-Droid main repository are Free Software.

This is a nonexhaustive list of what a reviewer would do:

- They will go to your source code repository, and look for copyright notices in license files, including *README*, to check that the proposed application is released under a recognized Free Software and/or OSI license(s).
- They will look at your build script to see which build system you use, and whether F-Droid build server can handle it (Ant and Gradle are the most common and easiest ones).

- They will try to download a copy of your source code.
- They will look in all source code files to verify that their licenses are consistent with corresponding license/README files.
- They will check if your application uses any pre-compiled libraries or binary blobs.
- They will look at your non-source code files to identify Non-Free resources used in your application.
- They will skim through the source code to see if your application uses Non-Free dependencies, shows advertisements, tracks users, promotes or depends on Non-Free or non-changeable services/applications, or does anything that is harmful or otherwise undesirable for users.
- They will list a summary of any *AntiFeatures* in your application.
- They will try patching your application to remove usage of third-party proprietary software (if there is any).
- They will try to determine a suitable update process for your application (e.g. by looking at how your releases relate to VCS tags and/or version information in *AndroidManifest.xml*, build.gradle.kts, pubspec.yaml or elsewhere).
- They will try writing a suitable metadata file for your application, and add it to local F-Droid build server instance. (`fdroid rewritemeta` , `fdroid lint` are used to ensure that metadata is well-formed)
- They will try to build your application in an isolated environment to see if the process succeeds and yield a functional APK.
- Usually the Gitlab CI will be used, but for apps that need more resources *(storage, memory, time)* than what the runners can offer, they might use local machines to prepare and test the metadata.
- If all went smoothly, they will add a new metadata file to their local *fdroiddata* git repository and synchronizes the change to GitLab.

In the case that the application failed some steps in the review, feedback will be given in the original submission queue thread where the proposal was posted.

Once the *fdroiddata* repository is updated on GitLab, it's mostly just a matter of time before F-Droid's official build server will fetch, build, and publish your application on the main F-Droid repository.

You can confirm the inclusion of your application by looking at the GitLab *fdroiddata* revision history.

Build Process

After the application metadata is added to *fdroiddata* GitLab repository, the next step is for the main F-Droid build server to fetch the applications source code and related components, build the application, and publish it on the main F-Droid repository.

This build process is not running on a schedule, a cycle is started after the previous one has been published, applications are processed in batch *(you can infer the average frequency by looking at past cycles)*. As steps are done behind the scene and are mostly automatic; all the submitter needs to do is to wait for it to finish.

A record of a successful build process for one application is provided on the F-Droid's website page for that specific app (e.g. see the Build Log for the F-Droid Client).

Apps that fail will have the log available during the build cycle on the F-Droid Monitor - Running page or, if in the previous cycle, on the Build page. This is useful to aid in diagnosing problems when the build unexpectedly failed.

What to Expect

When your application metadata is approved and accepted into the *fdroiddata* git repository on GitLab, **it won't immediately appear** in the main F-Droid repository.

Provided that your application does not have any build problems, it would takes somewhere **around 24 to 48 hours** from *fdroiddata* merge until the application to appears in the main repository.1 This timing limitation is due to the APK signing part of the build process, which requires human intervention on keystore access step.2

After publishing, the apps are immediately available in the repository and any client can now update or install. The f-droid.org website takes time to generate pages for updated apps and new apps, for all the languages, so there's a slight delay until it gets updated.

External Links

- F-Droid application submission queue on GitLab (for new submissions)
- F-Droid application submission queue on forum (for following-up old submissions)
- *fdroiddata* GitLab repository
- *fdroiddata* revision history

Edit on GitLab